

Blueprint-автоматизации «Минитендер» в Kimi Work

Дата: 03.08.2026 · Источник данных: backend/db.sqlite3 (dev-БД Django)

Принцип: автоматизации **дополняют** backend, а не заменяют его.

Email заказчику о КП (B7), напоминания 24ч/2ч (B10), inbound-парсер (B1) и Celery-задачи (parse/match/send_rfq/geocode/discover/send_rfq_email) остаются в backend и здесь **не дублируются**.

1. Автоматизации

Название	Триггер	Действие	Статус
Монитор новых КП (automation_4148a73e-dfb2-4841-adc2-a8317a20d568)	condition, опрос каждые 15 мин: срабатывает только когда MAX(quotes.id) вырос (момент ≈ приход КП от поставщика, включая inbound email после парсера B1)	Артефакт с лентой КП → виджет «Лента КП» + desktop-уведомление оператору	☐ Включена. Запуск проверен, доставка в виджет и уведомление работают
Статусы заявок и протокол PDF (automation_fc3ebf99-6c60-4e93-962c-094708b3a782)	condition, опрос каждые 15 мин: срабатывает только когда (а) появился competitive_sheets по заявке с ранним статусом или (б) quote.status = 'selected' по незавершённой заявке	(а) статус заявки → ready («Ожидает решения»); (б) статус → completed + генерация протокола PDF (reportlab, кириллица) в protocols/. Есть режим dry_run	☐ Включена. Проверена e2e: RFQ-K8X3CA rfq_sent → ready → completed; RFQ-ZUABCR rfq_sent → completed + protocols/protocol_ZUABCR.pdf
Воронка заявок (automation_d32a2c8a-df0b-4fcc-bc5d-f5819baf4e04)	interval, каждые 30 мин (только чтение)	Артефакт «этап → количество заявок» → виджет «Воронка заявок»	☐ Включена. Запуск проверен, доставка в виджет работает

Логика «триггер → условие → действие»

Монитор новых КП

- Триггер: condition каждые 15 мин.
- Условие: MAX(quotes.id) больше сохранённого last_seen_quote_id (state-файл).
- Действие: выборка последних 10 КП (заявка, поставщик, сумма = Σ цена×кол-во + доставка), обновление state, артефакт в виджет + desktop-уведомление.
- Уведомление приходит только при реально новых КП — condition-триггер не запускает автоматизацию «вхолостую».

Статусы + протокол

- Триггер: `condition` каждые 15 мин.
- Условие А: существует `competitive_sheets` для заявки со статусом не из `(ready, completed, cancelled)`.
- Действие А: `requests.status = 'ready'`.
- Условие Б: существует `quotes.status = 'selected'` по незавершённой заявке.
- Действие Б: `requests.status = 'completed'` + PDF-протокол (победитель, сумма, состав КП, сроки, оплата).
- Протоколы сохраняются в `protocols/` (корень проекта) и в каталоге запуска автоматизации.
Регистрация файла в `run.files` хост-валидатором на Windows не проходит (Windows-путь с буквой диска распознаётся как URL) — поэтому стабильная копия в `protocols/`.
- Проверка без побочных эффектов: запуск с входом `{"dry_run": true}`.

Воронка заявок

- Триггер: `interval` каждые 30 мин.
- Условие: не требуется (read-only агрегация).
- Действие: `GROUP BY status` по 12 статусам pipeline → артефакт в виджет.

2. Дашборд «Минитендер · Процессы»

Canvas ID: `canvas_33bb9c53-f80c-4333-9847-b52e9012733c`

Компонент	Что показывает	Источник данных
Воронка заявок (<code>widget_ba93ddb-1ae2-4c3b-b5f9-ee521e9f37a</code> , слева)	Количество заявок на каждом этапе: Черновик → Распознаётся → ... → Ожидает решения → Завершена; исключения (Ошибка RFQ, Отменена) отдельными метками	Автоматизация «Воронка заявок» → таблица <code>requests</code> , <code>GROUP BY status</code>
Лента КП (<code>widget_cf9b0293-c1d1-4271-ba8a-1b9e8e3336f8</code> , справа)	Последние 10 КП: поставщик, код заявки, сумма, время; новые с прошлой проверки подсвечены	Автоматизация «Монитор новых КП» → таблицы <code>quotes</code> , <code>quote_items</code> , <code>request_items</code> , <code>suppliers</code>

Оба виджета адаптивные (`compact/regular/expanded`), работают в светлой и тёмной теме, до первых данных показывают помеченный иллюстративный пример.

3. Инструкция по подключению

Текущее состояние (уже активно)

1. Три автоматизации созданы, код проверен реальными запусками (`status: succeeded`).
2. Виджеты созданы и провалидированы, привязки (Binding) активны и в статусе `valid`.
3. Дашборд собран: оба виджета размещены 6×9 на Canvas «Минитендер · Процессы».
4. Триггеры включены: две condition-автоматизации (15 мин) и одна interval (30 мин).

Ориентировочная нагрузка: до ~96 опросов/день каждой condition-автоматизации (запуск — только при истинном условии) + 48 запусков/день воронки.

Изменить частоту: `Automation.update` → `trigger.every` (например `"30m"` / `"1h"`).

Ручной запуск в любой момент

- «Монитор новых КП» и «Воронка» — запуск без параметров.
- «Статусы + протокол»: безопасная проверка `{"dry_run": true}`, боевой запуск — без параметров.

Если backend переедет на PostgreSQL

Сейчас автоматизации читают dev-БД SQLite напрямую. Для prod:

- заменить `db_path` на доступ к PostgreSQL (нужен `psycopg`) **или**
- перевести автоматизации на чтение REST API backend (`/api/requests/`, `/api/quotes/`) с JWT-сервисным токеном. Входной параметр `db_path` вынесен в input-контракт.

Ограничения

- Прямая запись статусов идёт в SQLite; при работающем dev-сервере Django конфликтов не было (короткие транзакции, `busy_timeout`). Для prod — только через API.
- Регистрация протоколов в `run.files` недоступна на Windows (см. выше) — стабильные копии лежат в `protocols/`.